Exhaustive Investigation into Open-Weight and Open-Source Raster-to-Vector Conversion Systems

The translation of raster imagery—composed of discrete pixel grids—into scalable vector graphics (SVG, EPS, PDF) is a foundational problem bridging computer vision, computational geometry, and generative artificial intelligence. For decades, this domain was entirely dominated by classical, deterministic algorithms relying on edge detection, mathematical curve fitting, and morphological skeletonization. However, the ecosystem has recently fractured into three distinct paradigms. Alongside classical algorithmic tracers, the field has seen the rapid maturation of hybrid optimization pipelines driven by differentiable rasterization, and, most recently, purely neural autoregressive generators powered by Vision-Language Models (VLMs).
The following analysis presents an exhaustive, deeply researched investigation into the global ecosystem of open-source and open-weight raster-to-vector conversion systems. This report documents everything from ubiquitous industry-standard tracing libraries to obscure academic prototypes, ensuring maximum recall of the vectorization landscape.
To guarantee rigorous classification regarding the actual availability of each project, candidates are categorized under a strict taxonomic framework:
Type A (Fully open-source) indicates that the source code and all necessary execution components are available under recognized open-source licenses such as MIT, GPL, or Apache-2.0. Type B (Open-weight) signifies that the neural network model weights are publicly downloadable and usable locally, even if surrounding inference code contains minor restrictions. Type C (Open-source implementation + separately available model) denotes that the execution code is open, and required weights are provided separately but remain publicly downloadable. Type D (Research release / partially open) represents systems where the authors have released sufficient code or assets to run the system, but the release is geared purely toward academic replication, or the licensing remains ambiguous. Finally, Type E (Not qualifying) is reserved for projects that appear relevant but are strictly proprietary, API-only, closed-weight, or perform a completely different function, such as raster embedding or deep feature-vector extraction.

Master Analytical Matrix of Vectorization Systems

Project	Type	Architecture Paradigm	Openness	License	Input Formats	Output Formats	Optimal Domain	Local Execution	GPU Required	Model / Download Siz	Official Source / Reposit

										e	**ory**
OmniSVG	General	AI (VLM)	Type B	Apache-2.0	Raster Images, Text	SVG	Complex illustrations, anime characters, colored icons	Yes	Yes	3B, 4B, 8B variants (7GB - 26GB)	OmniSVG/OmniSVG
StarVector	General	AI (VLM)	Type B	Apache-2.0	Raster Images, Text	SVG	Technical diagrams, charts, black-and-white icons	Yes	Yes (CPU via GGUF)	1B, 8B variants	joanrod/star-vector
Potrace	General	Classical	Type A	GPL-2.0	PBM, PGM, PPM,	SVG, EPS, PDF	High-contrast typogr	Yes	No	< 5 MB	potrace.sourceforge.n

					BMP		aphy, monochrome line art				et
AutoTrace	General	Classical	Type A	GPL-2.0 / LGPL-2.1	PNG, BMP, TGA, PNM	SVG, EPS, DXF	Centerline tracing, CAD outlines	Yes	No	< 10 MB	autotrace/autotrace
VTracer	General	Classical	Type A	MIT	PNG, JPG, BMP	SVG	High-res scanned blueprints, retro pixel art, colored photos	Yes	No	< 10 MB	visioncortex/vtracer
Geom	Artistic	Classic	Type A	MIT /	PNG,	SVG,	Artistic	Yes	No (GP	< 20 MB	Tw1ddl

etrize		al		GPL	JPG, GIF	JSON	abstraction, low-poly aesthetics		U forks exist)		e/geometrize
DiffVG	Renderer	Hybrid	Type D	Apache-2.0	Raster target	Differentiable paths	Backend optimization framework for machine learning	Yes	Yes	N/A (Codebase)	BachiLi/diffvg
Bézier Splatting	Renderer	Hybrid	Type D	MIT	Raster images	SVG	High-fidelity vector reconstruction,	Yes	Yes	N/A (Codebase)	xiliu8006/Bezier_splatting

							scalable dataset generation				
Im2Vec	General	AI (Latent)	Type D	MIT	Raster images	Continuous vectors	Research, simple vector synthesis without vector supervision	Yes	Yes	N/A	preddy5/Im2Vec
CLIPasso	Artistic	Hybrid	Type D	MIT	Photographs	Bézier curves	Semantic object sketching, abstraction	Yes	Yes	Dependent on CLIP	yael-vinker/CLIPasso
Deep	Domai	Hybrid	Type D	MIT	Hand-d	SVG	Resolvi	Yes	Yes	< 1 GB	Nauhcn

Sketch	n				rawn sketches		ng messy, overlapping hand-drawn strokes			(Light/Full)	ay/Deep-Sketch-Vectorization
SLD-Vectorization	Domain	Classical	Type D	MIT	Single-line drawings	SVG	Continuous single-stroke extraction	Yes	No	N/A (Codebase)	tanguymagne/SLD-Vectorization
FloorplanTransform	Domain	Hybrid	Type D	MIT	Raster floorplans	Vector text / 3D	Architectural CAD generation	Yes	Yes	Unknown	art-programmer/FloorplanTransformation
PolyVecto	Domain	Hybrid	Type D	Custom/Ac	Line drawin	SVG	Resolving	Yes	No (CPU	N/A (Codeb	ivanpuhach

r Flow				ademic	gs		complex stroke intersections and junctions		heavy)	ase)	ov/line-drawing-vectorization
Virtual Sketching	Domain	Hybrid	Type D	Apache-2.0	Rough sketches, photos	.npz vector arrays	Sketch cleanup, photograph-to-line-art translation	Yes	Yes	~250 MB	MarkMoHR/virtual_sketching
IconShop	Domain	AI (Transformer)	Type D	Custom/Academic	Text	SVG	Monochrome vector icon synthesis	Yes	Yes	Unknown	OmniSVG (GitHub)
Dee	Do	AI	Typ	Aca	Ras	SV	Hier	Yes	Yes	Unk	alex

pSVG	main	(Generative)	e D	demic	ter icons	G	archical vector animation and interpolation			nown	andre01/depsvg
libdepixelize	Domain	Classical	Type A	GPL-3.0	Pixel art	SVG	Extracting smooth splines from 8-bit/16-bit retrographics	Yes	No	< 5 MB	vvaniru dh/Pixel-Art
Imagetracerjs	Library	Classical	Type A	The Unlicense	PNG, JPG	SVG	Client-side, browser-based	Yes	No	< 1 MB	jankovicsandras/imagetracerjs

							vectorization				

1. Artificial Intelligence and Neural Vectorizers

The most profound recent breakthrough in raster-to-vector conversion bypasses geometric heuristics entirely. Instead of attempting to trace pixels or fit mathematical curves via localized edge detection, modern neural vectorizers treat SVG generation as a code-synthesis problem or a direct latent-space mapping challenge. These models ingest a raster image and output sequences of discrete SVG tokens, fundamentally bridging the gap between pixel perception and programmatic geometry.

OmniSVG represents the current state-of-the-art in end-to-end multimodal SVG generation[1]. Built upon the Qwen2.5-VL foundation model, OmniSVG completely reframes image vectorization as a sequence-to-sequence translation task[4]. The architecture tokenizes both textual and visual inputs as prefix tokens, while a specialized SVG tokenizer encodes vector graphics commands into a unified representation space[4]. Unlike earlier neural models that struggled with spatial precision by treating coordinates as plain text strings, OmniSVG parametrizes atomic SVG path commands into discrete tokens[1]. The system was trained on MMSVG-2M, a massive proprietary dataset containing two million annotated SVG assets, meticulously split across icons, illustrations, and complex anime characters[1]. To eliminate the ambiguities of highly nested XML, the training data was simplified down to fundamental commands: MoveTo (M), LineTo (L), Cubic Bézier (C), Elliptical Arc (A), ClosePath (Z), and Fill (F)[4]. This allows the model to excel at generating complex, multi-colored SVGs, surpassing traditional bounds that previously limited neural generators to monochrome, over-simplified icons[1]. Released as an open-weight project (Type B) under the Apache-2.0 license, the models are available on Hugging Face in 3B, 4B, and 8B parameter variants[1]. The 8B model requires approximately 26 GB of VRAM for inference and demonstrates varying inference times based on token length, scaling from 5 seconds for 256 tokens up to nearly 100 seconds for 4096 tokens[7].

StarVector operates as another premier multimodal vision-language model explicitly fine-tuned to transform image vectorization into a programmatic code-generation task[8]. The architecture pairs a robust vision encoder (such as SigLIP or CLIP) with an autoregressive language decoder based on StarCoder2 or BigCode[8]. Rather than producing an intermediate continuous vector representation, StarVector predicts native SVG text tokens directly[8]. The system was trained on SVG-Stack, a massive dataset encompassing 2.1 million curated samples, and its performance was evaluated against a custom benchmark suite known as SVG-Bench[8]. StarVector is highly adept at processing technical diagrams, icons, logotypes, and charts[8]. It produces highly compact, semantically meaningful path structures rather than the heavily fragmented paths

typical of traditional tracers like VTracer[11]. However, the model exhibits clear limitations regarding natural images or highly complex colored illustrations, as these fall outside its training distribution[8]. Furthermore, StarVector is bounded by an 8k token context window, meaning it can struggle to process highly intricate SVGs that require extensive coordinate mapping, an area where OmniSVG's 16k context window provides a distinct advantage[4]. The StarVector models are available as open-weight (Type B) releases in 1B and 8B parameter sizes via Hugging Face[8]. The project features a highly optimized Rust inference backend (starvector-rs) supporting GGUF formats for CPU execution, alongside VLLM and PagedAttention support for high-throughput GPU inference in production environments[8].

Im2Vec, introduced at the Computer Vision and Pattern Recognition (CVPR) conference in 2021, represents a pioneering neural architecture designed for synthesizing continuous vector graphics without requiring direct vector supervision[14]. Im2Vec trains neural networks to map raster pixels directly to vector parameters using a continuous latent space[16]. It operates by reconstructing the image via a differentiable rendering pipeline during training, allowing the network to infer vector shapes purely from raster examples[16]. While highly influential for establishing the viability of unsupervised vector generation, it struggles with out-of-domain generalization compared to modern autoregressive VLMs[15]. Im2Vec is available as a Type D research prototype, providing Python source code under an MIT license[14].

DeepSVG operates as a hierarchical generative network tailored specifically for complex SVG icon generation, animation, and latent space interpolation[18]. The architecture effectively disentangles high-level shapes from the low-level commands that encode the shapes themselves, predicting a set of shapes in a non-autoregressive fashion[19]. This allows the model to perform complex latent space algebra, smoothly interpolating between distinctly different vector graphics[18]. The project is classified as a Type D research release, with its PyTorch implementation and associated datasets hosted on GitHub[18].

IconShop functions as an autoregressive transformer designed exclusively for text-guided vector icon synthesis[22]. While heavily focused on text-to-SVG rather than pure image-to-SVG, it establishes foundational methodologies for tokenizing vector graphics that heavily influenced later models like OmniSVG[4]. IconShop treats SVG coordinates as number strings and generates monochrome, icon-level SVGs[3]. It is available as an academic release (Type D)[22].

SVGDreamer and SVGFusion represent the application of diffusion models to vector graphics generation. SVGDreamer leverages text-guided SVG generation by abstracting pixel-based diffusion models, using Score Distillation Sampling (SDS) to guide the placement of vector paths[14]. SVGFusion extends this by proposing a scalable text-to-SVG generation pipeline via vector space diffusion[22]. Both models operate as Type D research prototypes, showcasing the potential of diffusion mechanisms in generating primitive-based imagery[25].

2. Classical and Algorithmic Vectorizers

Despite the rapid ascent of neural networks, classical algorithmic vectorizers remain the undisputed standard for production environments. These systems rely on deterministic mathematics, morphological operations, graph theory, and heuristic curve-fitting. Their

absolute predictability, execution speed, resolution independence, and lack of specialized hardware dependencies make them indispensable for high-volume archiving, typography, and computer-aided design (CAD).

Potrace, developed by Peter Selinger, is arguably the most recognized and heavily utilized open-source vectorizer in existence[27]. It serves as the foundational backend for the tracing engines in Inkscape, FontForge, and countless proprietary design applications[27]. Potrace operates strictly on binary (black and white) bitmaps[28]. The algorithm processes the image by decomposing the raster into boundary paths, approximating these paths with optimal polygons, and finally smoothing the polygons into highly precise Bézier curves[27]. It is peerless for processing high-resolution typographic scans, black-and-white logos, and clean line art where exact geometric fidelity is paramount[27]. Because it fundamentally relies on outline tracing—tracing the outer boundary of a shape rather than its internal skeleton—it requires wrappers or preprocessing scripts to handle multi-colored images, typically by segmenting the image by color and passing each layer to Potrace independently[29]. Potrace is fully open-source (Type A) under the GPL-2.0 license, written in C, and boasts bindings in virtually every modern programming language[27].

AutoTrace, originally authored by Martin Weber, serves as a powerful counterpart to Potrace[32]. While Potrace excels at outlining solid shapes, AutoTrace was explicitly designed with robust centerline tracing capabilities[32]. It can extract the mathematical skeleton of a raster stroke, reducing a thick, hand-drawn line into a single, zero-width vector path[32]. This specific capability is a critical requirement for CAD applications, CNC machining, laser cutting, and animation inbetweening[32]. AutoTrace supports internal color reduction, despeckling, and outputs to an exceptionally wide array of legacy and modern formats, including SVG, EPS, PDF, DXF, and CGM[32]. The software is fully open-source (Type A), licensed under GPL-2.0 for the main executable and LGPL-2.1 for the underlying library[32]. Although the original repository saw periods of inactivity, modern forks maintained by the community ensure compatibility with contemporary compilers and updated ImageMagick dependencies[32].

VTracer, engineered in Rust by the Visioncortex team, is a modern algorithmic pipeline designed specifically to address the architectural limitations of Potrace—most notably its struggles with colored images and extreme gigapixel resolutions[34]. VTracer actively eschews Potrace's computationally expensive $O(n^2)$ optimal-polygon search[34]. Instead, it utilizes a fast, linear processing pipeline combined with hierarchical watershed clustering operating on the pixel graph[34]. This sophisticated approach allows VTracer to form edge-aware regions for incredibly sharp color segmentation[34]. Consequently, it natively handles multi-colored raster images, generating seam-free mosaics (gapless tessellations with shared boundaries) that entirely avoid the sub-pixel rendering gaps and overlapping artifacts common in naive color tracers[34]. The system also features adaptive Bradley-Roth thresholding, making it highly resilient when processing unevenly lit document scans or photographs[34]. VTracer is classified as Type A, licensed under MIT, and is distributed as a command-line interface, a Python binding, and a WebAssembly module[34].

Geometrize implements a fundamentally different philosophy regarding vectorization, functioning less as a literal conversion utility and more as a generative artistic abstraction tool[36]. Based heavily on Michael Fogleman's earlier primitive library, Geometrize approximates raster images using discrete geometric primitives such as circles, triangles, rotated rectangles, and Bézier curves[36]. The algorithm utilizes a hill-climbing optimization methodology[37]. In practice, it generates a massive volume of random candidate shapes, mathematically assesses their impact on reducing the pixel-wise delta between the current vector canvas and the target raster image, and commits the shape that minimizes the overall error[37]. This iterative process generates highly stylized, low-poly, or abstract geometric representations of photographs[36]. Geometrize is fully open-source (Type A) and is available across multiple languages, including a C++ desktop application, a Haxe port (geometrize-haxe), and a PHP implementation (geometrize-php)[36]. Furthermore, community forks such as zjl88858/forza-painter-geometrize-gpu have successfully implemented OpenCL and Vulkan GPU acceleration, resulting in massive execution speedups during the candidate evaluation phase[40].

Imagetracerjs, created by András Jankovics, provides a lightweight, entirely dependency-free raster-to-SVG tracer written in JavaScript, with a parallel implementation in Java (imagetracerjava)[41]. The algorithm performs initial color quantization, generates a discrete palette, and then traces the resulting contiguous color regions[41]. It applies straight-line control point radius adjustments and selective Gaussian blur to dynamically smooth out input noise[41]. Imagetracerjs is immensely popular for browser-based, client-side vectorization tasks where deploying a full backend processing server is undesirable[42]. It is classified as Type A, released into the Public Domain under The Unlicense[42].

Libdepixelize implements the groundbreaking Kopf-Lischinski algorithm, which fundamentally redefined the vectorization of pixel art[43]. Standard vectorizers like Potrace indiscriminately round off sharp corners, destroying the blocky aesthetic of 8-bit and 16-bit retro graphics. The Kopf-Lischinski algorithm instead resolves pixel connectivity graphs using a similarity heuristic to deduce the intended topology of the sprite[43]. It successfully distinguishes between intended sharp corners and intended smooth curves, extracting resolution-independent B-splines that perfectly upscale low-resolution game assets without blurring[43]. Available under the GPL-3.0 license across various repositories, it is a Type A project essential for retro-gaming upscaling pipelines[43].

3. Hybrid Systems and Differentiable Rasterization Pipelines

Hybrid systems attempt to merge the parameterization of classical geometry with the semantic optimization capabilities of neural networks. The absolute lynchpin of this category is the concept of Differentiable Rasterization. Standard classical rasterization operates as a discrete step function: a pixel is either inside a curve or outside of it. This binary state is non-differentiable, preventing neural networks from calculating gradients. Differentiable rasterization solves this by computing gradients analytically based on distance fields and soft

anti-aliasing techniques, allowing a neural network to calculate exactly how shifting a Bézier control point by a fraction of a millimeter will alter the final pixel color of the rendered image.
DiffVG (Differentiable Vector Graphics) is not a standalone end-user application; rather, it is the foundational PyTorch library that makes nearly all modern hybrid vectorization possible[45]. Developed by Tzu-Mao Li et al., DiffVG translates SVG paths into a differentiable space[45]. Higher-level tools utilize DiffVG by initializing random SVG paths on a canvas, rendering them through the DiffVG engine, comparing the rendered output to a target raster image using perceptual losses (like LPIPS) or Mean Squared Error, and then using standard backpropagation to shift the vector paths until the render matches the target[46]. DiffVG is classified as a Type D research release under the Apache-2.0 license, and remains deeply embedded in academic machine learning environments[45].
LIVE (Layerwise Image Vectorization) was developed to address severe topological failures inherent to DiffVG's optimization process[47]. When DiffVG attempts to optimize hundreds of curves simultaneously to match a complex image, the curves frequently fall into local minima, tangling into chaotic, intersecting arrangements that fail to visually cohere[47]. LIVE solves this vanishing gradient issue by progressively adding paths layer by layer[47]. It freezes earlier optimized paths and only allows newly introduced paths to optimize and fill in the residual visual errors[47]. This coarse-to-fine strategy ensures a clean, hierarchical SVG topology[47]. It is available as a Type D academic release.
Bézier Splatting represents a highly modern response to the extreme computational bottleneck of DiffVG[17]. DiffVG is exceptionally slow because it requires computationally intensive boundary sampling and complex gradient computations for anti-aliasing[17]. Vectorizing a high-resolution image using standard DiffVG can take several hours[17]. Bézier Splatting bypasses this by sampling 2D Gaussian distributions along the trajectory of Bézier curves and their interior regions[17]. Because Gaussians are naturally differentiable (a property heavily utilized in 3D Gaussian Splatting), the rasterizer inherently provides direct positional gradients at object boundaries[17]. This mechanism yields up to a 150x speedup in backward computation for open curves compared to DiffVG[17]. Furthermore, it employs an adaptive pruning and densification strategy that dynamically removes redundant curves and spawns new ones in areas of high loss, effectively escaping the local minima that plague static-curve optimization[17]. The project is a Type D release under the MIT license, requiring CUDA acceleration[49].
CLIPasso is a semantic object sketching system that relies entirely on DiffVG to generate abstract line-art vectors from photographs[46]. Instead of seeking pixel-perfect mathematical reconstruction, CLIPasso leverages the CLIP vision-language model to enforce semantic similarity[46]. It initializes a sparse set of Bézier curves and optimizes their parameters using DiffVG[46]. The loss function continuously compares the intermediate feature activations of the CLIP model for both the target photo and the evolving vector sketch[46]. This produces highly abstracted, semantic vector sketches whose level of detail is controlled simply by constraining the total number of strokes allowed during the optimization loop[46]. It is a Type D research prototype[46].

4. Libraries and Command-Line Tools

For developers and systems architects, vectorization functionality is frequently required as a programmatic building block rather than a standalone application. The open-source community maintains robust wrappers for this purpose.

The Potrace CLI and its associated bindings form the backbone of automated tracing. PyPotrace provides a seamless Python interface to the core C library, allowing it to be dropped into broader computer-vision pipelines without resorting to subprocess calls[31].

VTracer provides a similar programmatic surface area through vtracerpy, a pip-installable Python binding that exposes VTracer's fast, multi-color clustering algorithms natively to Python scripts[35]. Additionally, the core Rust implementation provides a compiled vtracer-cli binary that handles high-resolution processing directly from the terminal[34].

SQIP (SVG-based Low-Quality Image Placeholders) represents a highly specialized application of vectorization algorithms[50]. Rather than vectorizing an image for printing or CAD precision, SQIP utilizes the Geometrize/Primitive engine to create a tiny, sub-1KB SVG abstraction of a photograph[51]. This minuscule SVG is embedded directly into HTML payloads, allowing websites to display a recognizable, smooth geometric approximation of an image instantly while the high-resolution raster asset loads asynchronously in the background[51]. Implementations exist in Node.js, including Webpack loaders (sqip-loader)[52].

Geometrize itself is heavily modularized for programmatic use. The geometrize-haxe library allows the hill-climbing algorithm to be compiled across multiple targets, while geometrize-php brings the exact same geometric primitive generation directly to backend web servers running PHP[37].

5. GUI Applications

For non-programmatic users, several standalone graphical interfaces abstract the complexities of underlying vectorization algorithms.

ComfyUI-ToSVG bridges the gap between generative AI image synthesis and classical vectorization[53]. As a custom node extension for ComfyUI—a popular node-based backend for Stable Diffusion—it wraps the VTracer and Potrace algorithms directly into the generative pipeline[53]. This enables users to generate a raster image using a diffusion model, instantly pipe it into the vectorization node, and output a clean SVG string, all within a single workflow[53]. The node exposes granular control over VTracer parameters, including color precision, gradient step values, and curve simplification thresholds[53].

Image-to-SVG is a modern, self-hostable web application functioning as a dedicated graphical frontend for VTracer[54]. Developed using Svelte for the frontend and FastAPI for the Python backend, it is fully Dockerized for simple local deployment[54]. It provides a drag-and-drop interface supporting batch conversion and real-time progress tracking[54]. Crucially, it abstracts VTracer's complex parameters into accessible quality presets (High Quality, Balanced, Fast) that automatically tune the underlying thresholding, iteration counts, and speckle filtering variables[54].

The Geometrize Desktop Application is a compiled C++/Qt application that provides a highly

interactive workspace for generating geometric abstractions[36]. It allows users to manually restrict which shapes the algorithm is allowed to use, pause the hill-climbing process in real-time, and export the resulting coordinate data to SVG, PNG, or JSON formats[36].

6. Research Prototypes

The academic sector continuously produces experimental systems that push the theoretical boundaries of vector synthesis, though these systems often lack the polish of consumer software.

VectorNet approaches vectorization through the lens of semantic segmentation[55]. By utilizing neural networks, VectorNet analyzes a line drawing and attempts to segment it into distinct strokes and paths based on contextual understanding rather than pure pixel contiguity[55]. The repository provides TensorFlow implementations designed to process datasets like Quick Draw and Kanji characters[55].

SVGDreamer is an advanced text-guided SVG generation framework[25]. It utilizes a Diffusion Model backend to synthesize vector graphics, incorporating specialized loss functions to ensure the resulting SVGs maintain editable, distinct topological properties rather than dissolving into excessive, overlapping paths[22]. It is explicitly designed to advance the editability and diversity of text-to-SVG outputs[22].

7. Domain-Specific Vectorizers

Vectorization requirements vary wildly depending on the visual characteristics of the input medium. A tool calibrated for extracting high-contrast typography will fail catastrophically when applied to a manga page heavily shaded with halftone screentones.

Deep Sketch Vectorization via Implicit Surface Extraction was specifically engineered to handle the chaotic nature of hand-drawn sketches[56]. Traditional tracers fail on raw sketches because overlapping pencil strokes, varying pressures, and junction artifacts confuse edge-detection algorithms, resulting in fragmented vector paths. This SIGGRAPH 2024 project reframes sketch vectorization entirely as a surface extraction task[56]. A neural network predicts an Unsigned Distance Field (UDF) from the input sketch, estimating the true intended centerline of the messy strokes[56]. It then employs a Dual Contouring algorithm on this field to extract clean, topologically accurate vector lines, effectively ignoring the scribbles and noise inherent to human drawing[56]. The system is available as an MIT-licensed PyTorch implementation[56].

Virtual Sketching (General Virtual Sketching Framework for Vector Line Art) is a comprehensive framework tailored for vectorizing line art and simplifying rough sketches[57]. It utilizes a neural renderer paired with a perceptual model[57]. Uniquely, the system generates strokes dynamically, attempting to mimic human drawing order[57]. It outputs an intermediate .npz format that can be programmatically converted to SVG[57]. However, the authors explicitly note that standard SVG formats cannot natively replicate their custom dynamic-thickness stroke endpoints, meaning the SVG conversion is an approximation of the network's true output[57]. The repository provides pre-trained weights for clean line drawings, rough sketches, and photograph-to-line-art translation[57].

Single-Line Drawing Vectorization (SLDvec) targets the incredibly specific domain of

continuous single-stroke line drawings[58]. In these drawings, the line intersects itself multiple times, making it computationally difficult to recover the correct drawing order and topology[59]. SLDvec resolves this by fitting a single parametric curve—represented as a continuous Bézier spline—to the entire input raster[59]. The implementation relies on Python, but critically includes a C++ accelerated Voronoi diagram pruning module to efficiently compute the medial axes of the strokes[58].

PolyVector Flow (Keypoint-Driven Line Drawing Vectorization) focuses on resolving junctions (corners and intersections) in line drawings[60]. Traditional vectorizers often degrade sharp intersections into messy, high-degree vertex clusters[61]. PolyVector Flow addresses this by utilizing a PyTorch network to predict semantic keypoints across the raster drawing[60]. A subsequent C++ polyvector flow algorithm leverages these keypoints to cleanly resolve the stroke intersections, constructing a rigorous planar straight-line graph[60]. The system is heavily reliant on the Gurobi optimization solver[60].

FloorplanTransformation is a highly specialized computer-vision pipeline explicitly tailored for architectural floorplans[62]. Built upon a pose estimation network architecture, it detects wall junctions, corners, and room typologies in raw raster floorplans[62]. It converts these detected features into precise vector representations, outputting text formats that can be instantly extruded into 3D CAD popup models[62].

8. Historical and Abandoned but Usable Projects

The open-source ecosystem is replete with projects that, while no longer actively maintained, remain functionally viable and mathematically relevant.

Older versions of AutoTrace provide a fascinating case study in open-source archaeology. While modern forks exist, the original repository saw extensive historical patching. Issues surrounding MacOS 10.13 compatibility and compiler drift led to fragmented development, with various developers (like Martin Weber himself) submitting zip files containing critical fixes to memory leaks and metafile handlers[33]. These historical patches, stored in archival branches, remain vital for researchers attempting to compile the toolchain on legacy systems[33].

The original primitive project by Michael Fogleman, written in Go, serves as the historical ancestor to the entire Geometrize ecosystem. While Geometrize modernized the hill-climbing approach and ported it to C++ and Haxe, the original Go implementation remains a masterclass in generative geometric abstraction and is still functionally usable for command-line users willing to compile Go binaries.

9. Projects Investigated but Excluded

To ensure the strict integrity of this investigation, several highly visible projects that utilize keywords such as "SVG", "Img2Vec", or "Vectorization" were rigorously evaluated and subsequently excluded. These candidates failed the core requirement of locally transforming a raster image into a scalable vector graphics file.

Img2Vec / VectorFormer (christiansafka/img2vec): Despite its highly suggestive name, this project does not generate vector graphics. It is a PyTorch library that utilizes pre-trained convolutional neural networks (such as ResNet and AlexNet) to extract deep feature

embeddings from images[64]. These embeddings—mathematical vectors representing image semantics—are used for downstream machine learning tasks like similarity search, image clustering, and classification[64]. (Classification: Type E).

Aspose ImageVectorizer (aspose-com-gists/convert-image-to-vector): This project appears prominently in GitHub searches for raster-to-vector conversion[66]. However, analysis of the repository reveals it merely contains C# code snippets that act as thin wrappers for the proprietary Aspose cloud API[66]. The actual conversion engine is completely closed-source and cannot be downloaded or executed locally[66]. (Classification: Type E).

VectoSolve (vectosolve-image-to-svg): Hosted on the GitHub Actions marketplace, this project claims to offer AI-powered SVG vectorization[67]. Investigation confirms it is a remote execution wrapper. It requires a paid API key and executes on proprietary external servers, charging a monetary cost (credits) per image conversion[67]. (Classification: Type E).

DiffBMP: While sharing nomenclature and theoretical lineage with DiffVG, DiffBMP is a differentiable renderer designed to arrange and optimize *bitmap primitives* rather than vector paths[48]. It utilizes a custom CUDA kernel to manipulate thousands of raster sprites, outputting a layered .psd file of rasters rather than a resolution-independent SVG[48]. (Classification: Type E).

ScanTailor / Unpaper: These classical document processing tools are frequently mentioned in archival discussions alongside Potrace. They perform adaptive thresholding, margin-cropping, and morphological dewarping on scanned document rasters to prepare them for Optical Character Recognition (OCR)[68]. However, they strictly output cleaned, binary TIFF or PNG raster files, not vector graphics[68]. (Classification: Type E).

Works cited

1. OmniSVG/OmniSVG - Hugging Face, https://huggingface.co/OmniSVG/OmniSVG
2. [NeurIPS 2025] OmniSVG is the first family of end-to-end ... - GitHub, https://github.com/OmniSVG/OmniSVG
3. OmniSVG: A Unified Scalable Vector Graphics Generation Model, https://huggingface.co/papers/2504.06263
4. OmniSVG: A Unified Scalable Vector Graphics Generation Model, https://papers.nips.cc/paper_files/paper/2025/file/a510f05a574d4203ef3952973672fe2f-Paper-Conference.pdf
5. OmniSVG: A Unified Scalable Vector Graphics Generation Model, https://omnisvg.github.io/
6. OmniSVG: A Unified Scalable Vector Graphics Generation Model, https://arxiv.org/html/2504.06263v3
7. OmniSVG/OmniSVG1.1_8B - Hugging Face, https://huggingface.co/OmniSVG/OmniSVG1.1_8B
8. StarVector: Generating Scalable Vector Graphics Code ... - GitHub, https://github.com/joanrod/star-vector
9. oxide-lab/starvector-rs - GitHub, https://github.com/oxide-lab/starvector-rs
10. StarVector, https://starvector.github.io/
11. StarVector: A Multimodal Model for SVG Code Generation,

https://comfyui-wiki.com/news/2025-03-26-starvector-svg-generation
12. Generating Scalable Vector Graphics Code from Images and Text, https://openaccess.thecvf.com/content/CVPR2025/papers/Rodriguez_StarVector_Generating_Scalable_Vector_Graphics_Code_from_Images_and_Text_CVPR_2025_paper.pdf
13. Rendering-Aware Reinforcement Learning for Vector Graphics, https://arxiv.org/html/2505.20793v2
14. MarkMoHR/Awesome-Sketch-Synthesis: :books - GitHub, https://github.com/MarkMoHR/Awesome-Sketch-Synthesis
15. Pradyumna Reddy Chinthala preddy5 - GitHub, https://github.com/preddy5
16. yinyunie/3D-Shape-Analysis-Paper-List - GitHub, https://github.com/yinyunie/3D-Shape-Analysis-Paper-List
17. Bézier Splatting for Fast and Differentiable Vector Graphics Rendering, https://neurips.cc/virtual/2025/poster/117178
18. DeepSVG - Alexandre Carlier - GitHub Pages, https://alexandre01.github.io/deepsvg/
19. DeepSVG: A Hierarchical Generative Network for Vector Graphics, https://huggingface.co/papers/2007.11301
20. SVG-Icons8 - OpenDataLab, https://opendatalab.com/OpenDataLab/SVG-Icons8
21. GitHub - alexandre01/deepsvg: [NeurIPS 2020] Official code for the, https://github.com/alexandre01/deepsvg
22. svg-generation · GitHub Topics, https://github.com/topics/svg-generation
23. Autoregressive-Models-in-Vision-Survey/README.md at main, https://github.com/ChaofanTao/Autoregressive-Models-in-Vision-Survey/blob/main/README.md
24. OmniSVG - GitHub, https://github.com/OmniSVG
25. SVGDreamer: Text Guided SVG Generation with Diffusion Model, https://github.com/ximinng/SVGDreamer
26. SVGFusion: Scalable Text-to-SVG Generation via Vector Space, https://github.com/ximinng/SVGFusion
27. Potrace, https://potrace.sourceforge.net/
28. a polygon-based tracing algorithm - Potrace, https://potrace.sourceforge.net/potrace.pdf
29. Man page for potrace(1) - SourceForge, https://potrace.sourceforge.net/potrace.1.html
30. GitHub - btk/vectorizer: Potrace based multi-colored raster to vector, https://github.com/btk/vectorizer
31. Potrace - Browse /1.16 at SourceForge.net, https://sourceforge.net/projects/potrace/files/1.16/
32. AutoTrace - bitmap to vector graphics converter - GitHub, https://github.com/autotrace/autotrace
33. Review patches from Martin Weber · Issue #15 · autotrace ... - GitHub, https://github.com/autotrace/autotrace/issues/15
34. visioncortex/vtracer: Raster to Vector Graphics Converter - GitHub, https://github.com/visioncortex/vtracer

35. vtracerpy - PyPI, https://pypi.org/project/vtracerpy/
36. white_square_button: Geometrize is a desktop app that ... - GitHub, https://github.com/Tw1ddle/geometrize
37. GitHub - Tw1ddle/geometrize-haxe: :triangular_ruler, https://github.com/Tw1ddle/geometrize-haxe
38. GitHub - Tw1ddle/geometrize-twitter-bot, https://github.com/Tw1ddle/geometrize-twitter-bot
39. Geometrize PHP is a PHP library for recreating images with ... - GitHub, https://github.com/Cerdic/geometrize-php
40. forza-painter geometrize GPU Version - GitHub, https://github.com/zjl88858/forza-painter-geometrize-gpu
41. jankovicsandras/imagetracerjava: Simple raster image tracer and, https://github.com/jankovicsandras/imagetracerjava
42. jankovicsandras/imagetracerjs: Simple raster image tracer ... - GitHub, https://github.com/jankovicsandras/imagetracerjs
43. Releases · h00shi/libdepixelize - GitHub, https://github.com/h00shi/libdepixelize/releases
44. We implement the famous "Depixelizing Pixel Art" paper by ... - GitHub, https://github.com/vvanirudh/Pixel-Art
45. GitHub - BachiLi/diffvg: Differentiable Vector Graphics Rasterization, https://github.com/BachiLi/diffvg
46. yael-vinker/CLIPasso - GitHub, https://github.com/yael-vinker/CLIPasso
47. LIVE- Towards Layer-wise Image Vectorization (CVPR 2022 Oral), https://github.com/Picsart-AI-Research/LIVE-Layerwise-Image-Vectorization
48. DiffBMP: Differentiable Rendering with Bitmap Primitives, https://openaccess.thecvf.com/content/CVPR2026/papers/Hong_DiffBMP_Differentiable_Rendering_with_Bitmap_Primitives_CVPR_2026_paper.pdf
49. GitHub - xiliu8006/Bezier_splatting, https://github.com/xiliu8006/Bezier_splatting
50. SQIP - a pluggable image converter with vector support - GitHub, https://github.com/axe312ger/sqip
51. SQIP is a tool for SVG-based LQIP image creation written in go, https://github.com/denisbrodbeck/sqip
52. EmilTholin/sqip-loader: Loads images and exports tiny ... - GitHub, https://github.com/EmilTholin/sqip-loader
53. Yanick112/ComfyUI-ToSVG: Converts raster images into ... - GitHub, https://github.com/Yanick112/ComfyUI-ToSVG
54. Image to SVG Converter - GitHub, https://github.com/sserada/png-to-svg
55. Semantic Segmentation for Line Drawing Vectorization Using, https://github.com/byungsook/vectornet
56. GitHub - Nauhcnay/Deep-Sketch-Vectorization, https://github.com/Nauhcnay/Deep-Sketch-Vectorization
57. [SIGGRAPH 2021] General Virtual Sketching Framework for Vector, https://github.com/MarkMoHR/virtual_sketching
58. Official code for the paper "Single-Line Drawing Vectorization" · GitHub, https://github.com/tanguymagne/SLD-Vectorization

59. (PDF) Single-Line Drawing Vectorization - ResearchGate, https://www.researchgate.net/publication/396238058_Single-Line_Drawing_Vectorization
60. Keypoint-Driven Line Drawing Vectorization via PolyVector Flow, https://github.com/ivanpuhachov/line-drawing-vectorization-polyvector-flow
61. Stroke Vectorization via Involution Prediction, https://igl.ethz.ch/projects/involution-stroke-vectorization/stroke-vectorization-via-involution-prediction-vmv-2026-gerstner-et-al.pdf
62. Raster-to-Vector: Revisiting Floorplan Transformation - GitHub, https://github.com/art-programmer/FloorplanTransformation
63. autotrace-0.40.0-20180915-MacOS.zip requires MacOS 10.13 or, https://github.com/autotrace/autotrace/issues/17
64. christiansafka/img2vec: :fire: Use pre-trained models in PyTorch to, https://github.com/christiansafka/img2vec
65. vecsim-demo/SemanticSearch1k.ipynb at master · RedisAI ... - GitHub, https://github.com/RedisAI/vecsim-demo/blob/master/SemanticSearch1k.ipynb
66. Convert Image to Vector | Image to SVG | PNG to Vector - Github-Gist, https://gist.github.com/aspose-com-gists/e34baddc9fef3943d09e7ee932e89a48
67. VectoSolve - Image to SVG · Actions · GitHub Marketplace, https://github.com/marketplace/actions/vectosolve-image-to-svg
68. Bezier-dewarping/flattening algorithm · Issue #20628 - GitHub, https://github.com/darktable-org/darktable/issues/20628